



THE COPPERBELT UNIVERSITY
SCHOOL OF INFORMATION COMMUNICATION TECHNOLOGY (SICT)
CS301 – PROJECT MANAGEMENT
SYSTEM DESIGN SPECIFICATIONS

PROJECT NAME: TAX-OPOLOGY: THE PUBLIC GOOD GAME

GROUP NUMBER: 12

PROGRAMME: COMPUTER SCIENCE

YEAR OF STUDY: 3

PROJECT SUPERVISOR: MR. MUMBA

SIN	FIRST NAME	LAST NAME
23124656	ISAAC	TEMBO
23122916	DARIUS	CHUNGU
23138696	BENJAMIN	CHAAMBWA
23127482	LWANGA	MWABA

Approved: ,



Contents

INTRODUCTION.....	1
ARCHITECTURE DESIGN.....	2
High-Level Architecture Overview.....	2
CONTEXT DIAGRAMS AND DATA FLOW MODELS.....	4
System Context Diagram.....	4
Data Flow Diagram (Context level).....	5
DETAILED DESIGN.....	7
Class Diagram.....	7
Sequence Diagrams.....	8
Database Schema.....	9
Interface Design.....	10
Game logic design.....	13
Data persistence.....	15
CONCLUSION.....	17

INTRODUCTION

This document seeks to present the detailed system design for “Tax-opoly: The Public Good Game”, an educational digital board game developed to simulate tax compliance behaviour and demonstrate the collective benefits of taxation as investments in public goods. The design directly translates the functional and non-functional requirements of the project into a coherent, implementable software architecture.

In line with the SICT guidelines, this document focuses on the engineering decisions taken during the design phase. It identifies the major design alternatives considered, provides principled rationale for each choice made, and critically assesses the strengths and limitations of those decisions. Particular attention is given to why certain technologies, architectural patterns, and modelling approaches were selected over others, taking into account the project’s educational objectives, target users, time constraints, and the need for an engaging gaming experience.

The system has been designed using the Object-Oriented Analysis and Design (OOAD) approach supported by the Unified Modelling Language (UML). This choice was preferred over the Structured Analysis and Design (SAD) approach because OOAD naturally supports the modular, reusable, and extensible nature required for a game like Tax-opoly that simulates dynamic player interactions, shared state and evolving public infrastructure.

The selected tech stack is primarily Godot Engine to work in a browser. This was chosen after evaluating alternatives for its low learning curve, built-in visual editor for rapid scene prototyping, native high-level multiplayer support suitable for turn-based games, excellent performance on low-end hardware, and perfect alignment with OOAD principles.

ARCHITECTURE DESIGN

High-Level Architecture Overview

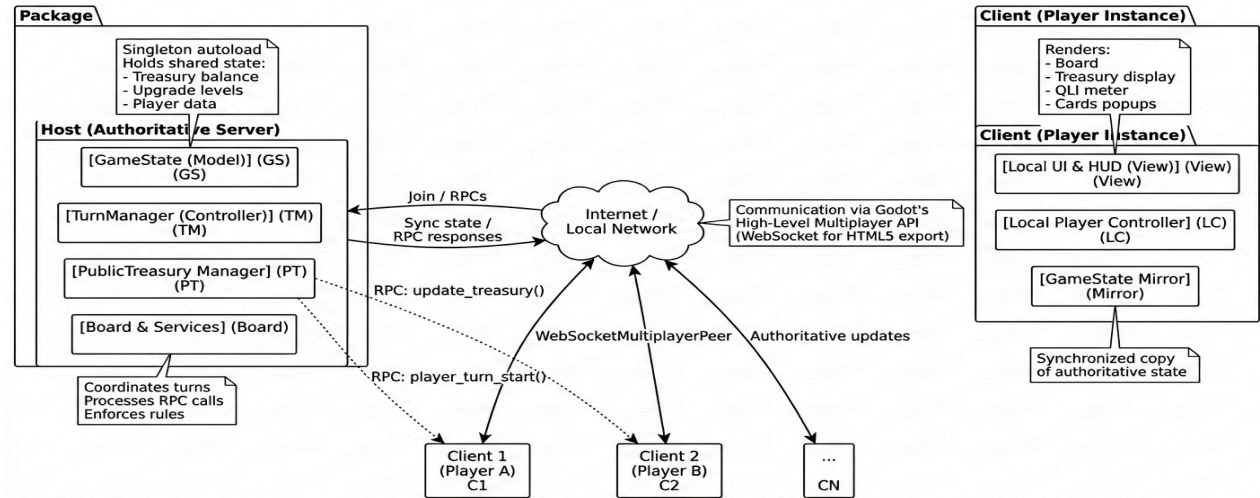
Tax-opoly adopts a client-server peer-to-peer architecture facilitated by Godot's High-Level Multiplayer API. One player acts as the host having server authority, while others join as clients. This model ensures authoritative control over shared game state such as Public Treasury balance, public service upgrade levels, turn order, and audit outcomes to prevent cheating in an educational simulation, while keeping latency low.

The system is structured around the Model-View-Controller (MVC) pattern, adapted to Godot's node-based composition:

- **Model:** The model includes core game logic and data such as game state singleton autoload, player resources, public treasury manager, board service nodes. It handles tax declaration, audit resolution, Quality of Life Index (QLI index) calculation, and state synchronization via Remote Procedure Calls (RPCs).
- **View:** The view includes the UI and visual representation of the game and its resources which include Heads Up Display (HUD) for personal savings or QLI, central Treasury display, animated board with upgrade levels, card popups. This will be implemented as Control nodes and CanvasLayer for responsive web-based scaling.
- **Controller:** The controller components of the design include input handling and orchestration i.e. TurnManager singleton coordinates player turns, processes inputs via RPC calls to the host, enforces rules like audit draws.

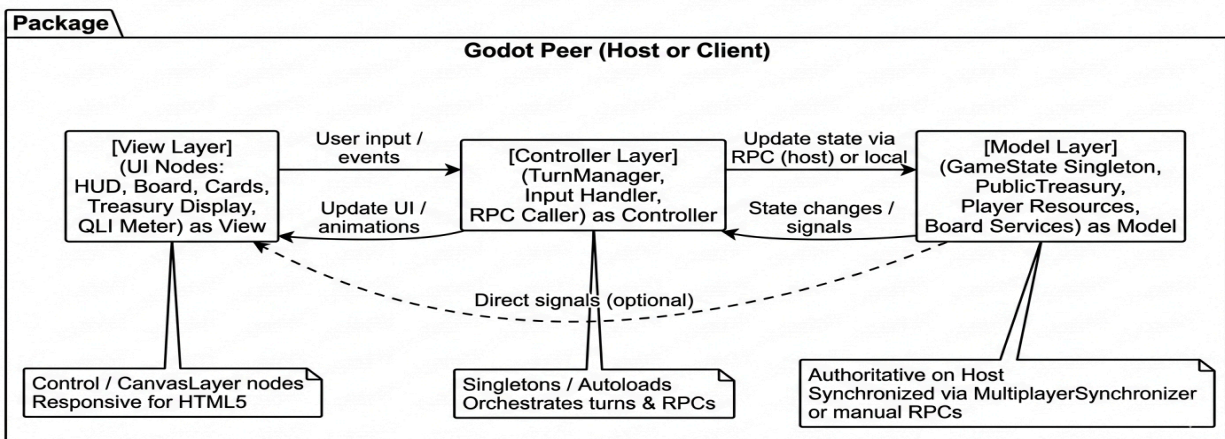
This MVC layering separates concerns: game rules remain in the Model (easy to test and debug), presentation in Views (visual polish), and flow in Controllers (clear turn progression).

High-Level Architecture of Tax-opoly (Godot Multiplayer)



This architecture diagram illustrates the foundational design pattern for Tax-opoly, showing how the system separates concerns through the Model-View-Controller (MVC) pattern adapted for Godot Engine. The diagram demonstrates the clear separation between game logic (Model), user interface (View), and input handling (Controller), which ensures maintainability and testability. The client-server peer-to-peer structure with host authority prevents cheating while maintaining educational integrity - crucial for a simulation designed to teach tax compliance behaviour. This design choice directly supports the game's educational objectives by ensuring consistent, authoritative game states across all players.

Adapted MVC Pattern in Tax-opoly (Godot Node Structure)



CONTEXT DIAGRAMS AND DATA FLOW MODELS

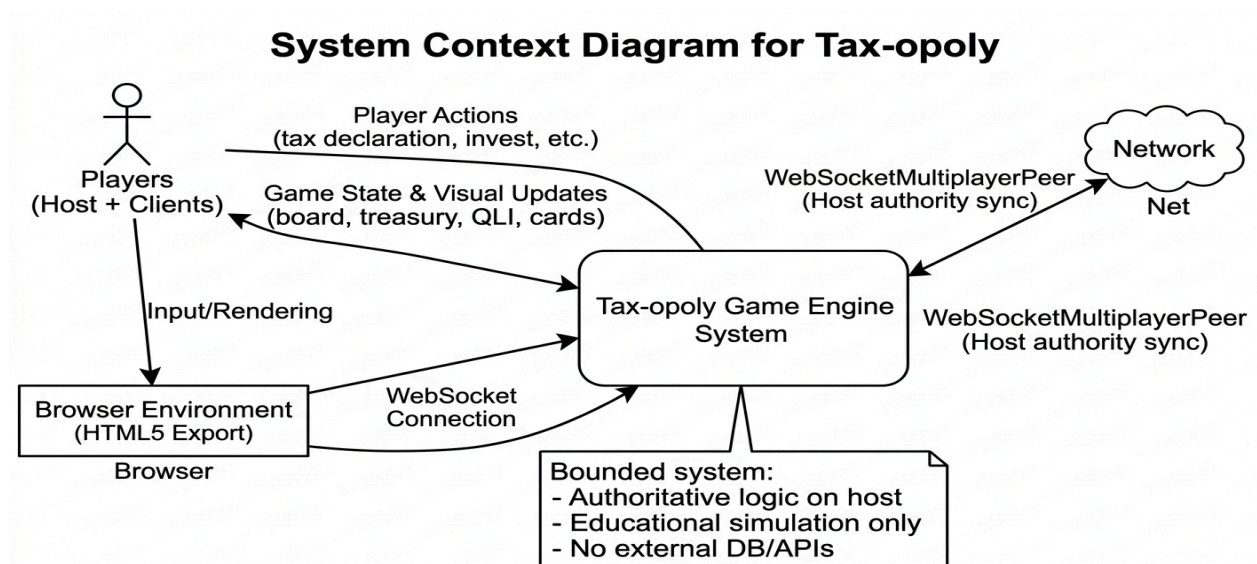
System Context Diagram

The System Context Diagram below provides a high-level overview of Tax-opoly as a bounded system and its interactions with external entities. It illustrates the primary actors (players) and any external interfaces, emphasizing the educational multiplayer nature of the application.

The central system is the Tax-opoly Game Engine implemented in Godot with multiplayer support. External entities include:

- Players: (1 host and up to 3 clients): Human users who interact via input devices such as keyboard, mouse, or touch screens. They send actions e.g., tax declaration, investment votes and receive game state updates e.g., board visuals, QLI scores.
- Browser Environment: For a web-based functionality provides rendering canvas, WebSocket connectivity, and local storage.
- Network: Facilitates peer-to-peer communication via Godot's WebSocketMultiplayerPeer.

No external databases, APIs, or hardware peripherals are required, keeping the system self-contained. The diagram highlights that all game logic and state authority reside within the system boundary, with the host peer enforcing rules to maintain educational integrity.



This context diagram defines the system boundary and external interactions for Tax-opoly, emphasizing its self-contained nature. The diagram shows how the game engine serves as the central hub, managing all interactions between players and the browser environment without requiring external databases or APIs. This design decision supports the educational goal by keeping the focus on gameplay mechanics rather than technical complexity. The peer-to-peer networking approach ensures low latency for turn-based gameplay while the browser-based deployment makes the game accessible to students across different platforms and devices.

Data Flow Diagram (Context level)

The Context Diagram is expanded slightly in the diagram below to show the major processes and key data flows.

Key processes at this level include:

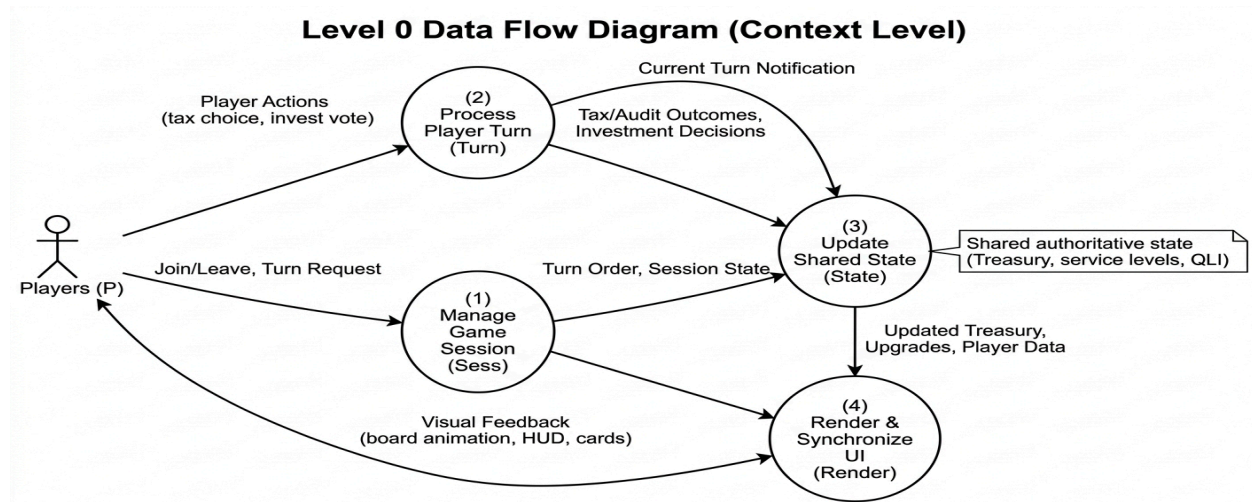
1. Manage game session: Lobby joining, turn synchronization, and multiplayer peer setup.
2. Process player turn: Handle tax declaration or under-declaration, audit card draws, investment proposals.
3. Update shared state: Modify Public Treasury, upgrade public services, calculate QLI.
4. Render and synchronize UI: Broadcast authoritative state to all clients for consistent visuals including board upgrades, treasury balance and card popups.

Major data flows:

- Player input (tax choice, investment decision) □ Process Player Turn.
- Audit/Risk outcomes □ Update shared state.
- State synchronization (treasury, upgrades, player savings) □ Render and synchronize UI □ Players.
- Turn Notification □ Players (signals whose turn it is).

This context diagram demonstrates how individual player decisions feed into collective outcomes (public goods upgrades), directly supporting the educational objective of illustrating

tax compliance benefits. It aligns with the MVC where inputs enter via Controller which then update Model, and refresh View across peers.



This data flow diagram expands on the context diagram to show how information moves through the Tax-opoly system during gameplay. The diagram illustrates the critical educational flow: individual player decisions (tax declarations, investment votes) feed into collective outcomes (public service upgrades), directly demonstrating the game's core teaching principle. The authoritative state management shown here ensures that all players see consistent results of their collective tax contributions, reinforcing the lesson that individual compliance contributes to shared public goods. The turn-based synchronization maintains engagement while allowing time for educational reflection on decisions.

These diagrams provide a clear, traceable view of system boundaries and information flows, facilitating verification against functional requirements such as multiplayer synchronization, collective investment.

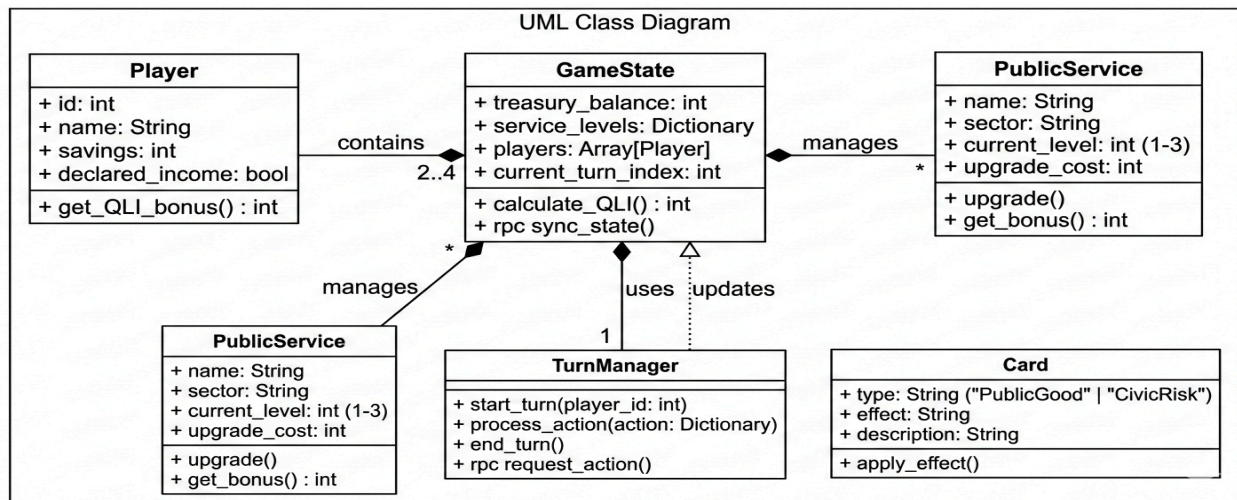
DETAILED DESIGN

The design leverages Godot Engine's node-based composition, where Nodes act as runtime objects, Scenes as composable class-like templates, Resources as data classes, and Autoload singletons as global managers. This approach ensures modularity, reusability, and clear mapping to OOAD principles such as inheritance via extends, composition via child nodes, encapsulation via signals and RPCs.

Class Diagram

The class diagram below illustrates the core classes in Tax-opoly. It uses Godot conventions:

- GameState - Manages shared authoritative state.
- Player - Represents each participant with personal data.
- PublicService - Board properties that can be upgraded.
- Card - For Public Good and Civic Risk cards.
- TurnManager - Controls turn sequencing and RPC orchestration.



This class diagram represents the object-oriented structure of Tax-opoly, showing the relationships between core game entities. The design follows Godot Engine conventions while

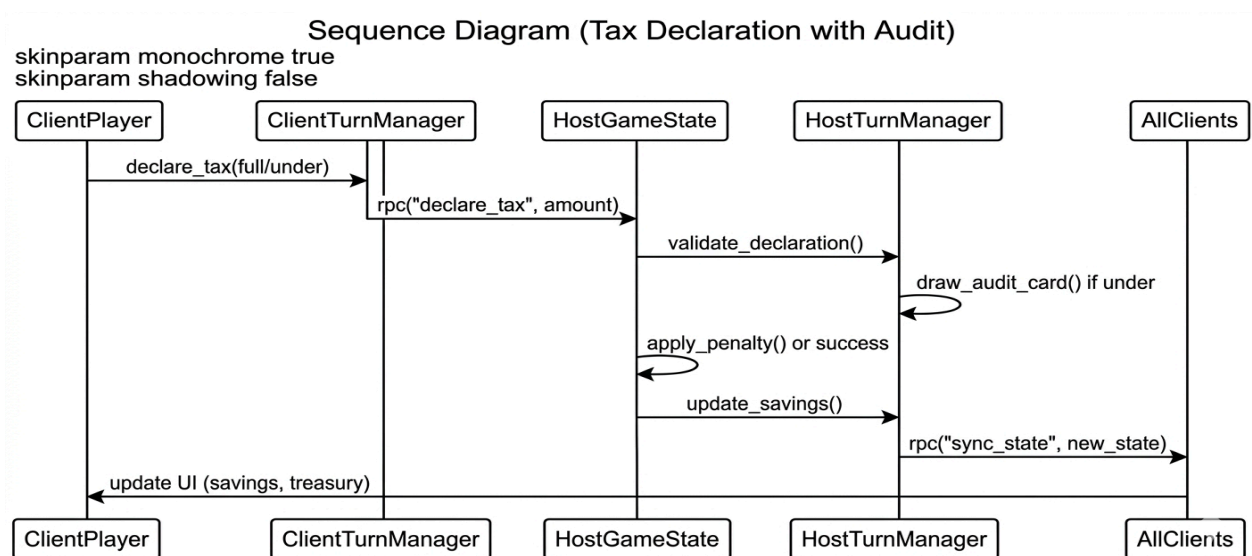
maintaining clear OOAD principles. Key relationships include the composition between GameState and Player objects, and the inheritance hierarchy for different card types. This structure supports extensibility - new card types or public services can be easily added without disrupting existing code. The clear separation of concerns between data management (GameState), player representation (Player), and game assets (PublicService, Card) facilitates both testing and future enhancements to the educational simulation.

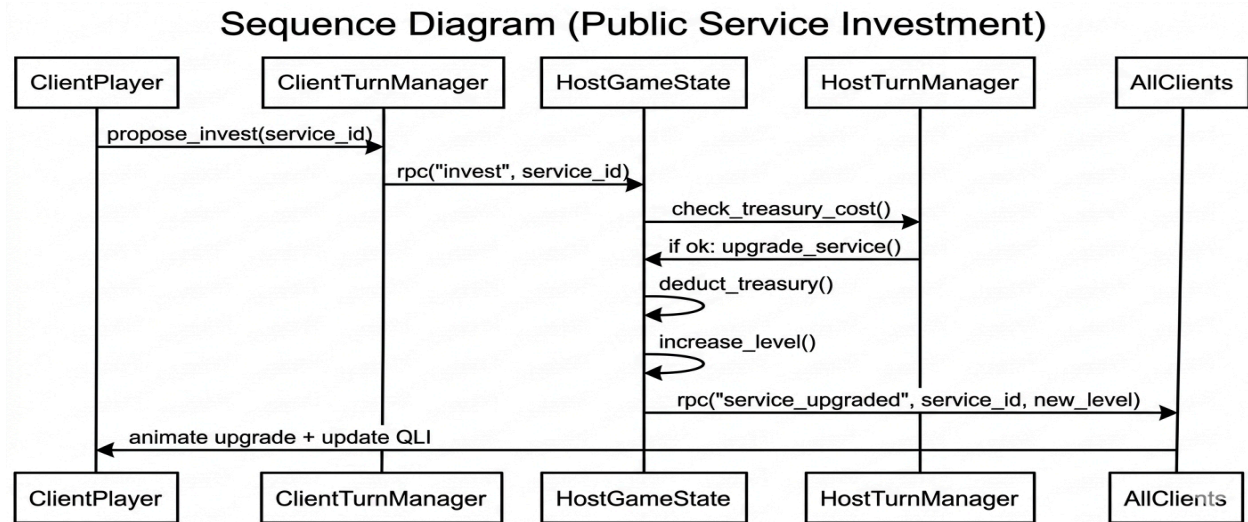
Sequence Diagrams

Two key interaction sequences are modeled:

1. Tax Declaration and Audit Flow: shows a client player declaring tax → RPC to host → host processes audit risk → syncs outcome to all clients. This demonstrates authoritative host model preventing cheating.
2. Public Service Investment Flow: player proposes investment → host checks treasury → upgrades service → broadcasts new level and QLI impact to all peers.

These sequences use Godot RPC notation (rpc(), rpc_id()) and signals for local UI updates.





These sequence diagrams detail the two most critical gameplay interactions in Tax-opoly: tax declaration with audit risk, and collective investment in public services. The first diagram shows how the host-authority model prevents cheating during tax declarations while introducing the educational element of audit risk. The second diagram illustrates the collaborative decision-making process for public investments, demonstrating how individual tax contributions translate into collective benefits. These interactions directly support the educational objectives by showing students both the risks of non-compliance and the benefits of collective tax contribution to public goods.

Database Schema

The Tax-opoly game is a purely runtime, multiplayer simulation with no persistent storage required for core gameplay as state resets per session. No traditional database is used.

- All state is held in memory via Godot autoloads and node properties.
- Transient data (player savings, treasury balance, upgrade levels) synchronized via Godot's MultiplayerSynchronizer nodes or manual RPCs.

This design choice reduces complexity and ensures that the game is browser compatible, has low resource use and focuses effort on educational mechanics.

Interface Design

The user interface of Tax-opoly is designed as a clean, board-game-style 2D experience that directly mirrors the Monopoly concept while being fully browser-responsive and accessible on low-spec devices. The interface follows Godot's Control-node and CanvasLayer system and is divided into the following screens:

1. Main Menu / Lobby Screen

- Simple start button, player-name input fields (up to 4 players), “Host Game” and “Join Game” buttons.
- Displays room code for easy local-network joining.

2. Game Board screen (Main play area)

This is shown in figure 1.4.4 as a 2D wireframe of the visualized look of the main play area which takes after the classic monopoly board look but replaces individual investment items used in monopoly with public goods that Tax-opoly seeks to use for its game play.

This main game play area contains:

- Central hexagonal or square board (inspired by Monopoly's property layout).
- 12–16 spaces representing public-service categories (e.g. Schools, Hospitals, Roads, Water Supply, Security).
- Each space shows the current upgrade level (visual icons that animate when upgraded).
- Players' tokens move clockwise around the board (turn-based, like Monopoly).
- Top HUD: Personal Savings, Tax Rate Slider (pay or evade decision), Current Income.
- Center panel: Shared Public Treasury balance + live Quality-of-Life Index (QLI) meter (bar or circular gauge that fills as collective contributions rise).
- Right sidebar: Player list with names, current wealth, and compliance status (green or red icons).

3. Decision / Card Popup Screens

A visualized mock up of this scene is shown below in figure 1.4.4.1 which gives the user a choice to make. To either pay the tax or evade it and also the card contains simple explanations to help the user understand the implications of their decisions. Some of the sections contained in this scene include:

- Large modal popups for tax declaration (slider + “Pay Full Tax” vs “Evasion Attempt” buttons).
- Civic Risk / Public Good Chance cards (exactly like Monopoly’s Chance & Community Chest) – animated card flip with explanatory text linking the outcome to tax morale or free-rider problem.

4. End-of-Game Summary Screen

- Final QLI score, total treasury collected, individual contributions ranking.
- “High Score” entry field for name (auto-filled from lobby).

All UI elements use Godot’s Theme system for consistent styling. Feedback is immediate: colour-coded messages (“Tax Paid – + QLI!” or “Audit Failed – Penalty!”) and sound cues. The design ensures a minimal learning curve, satisfying the usability and educational-requirements sections of the Requirements Specifications.

Tax-opoly Main Game Interface

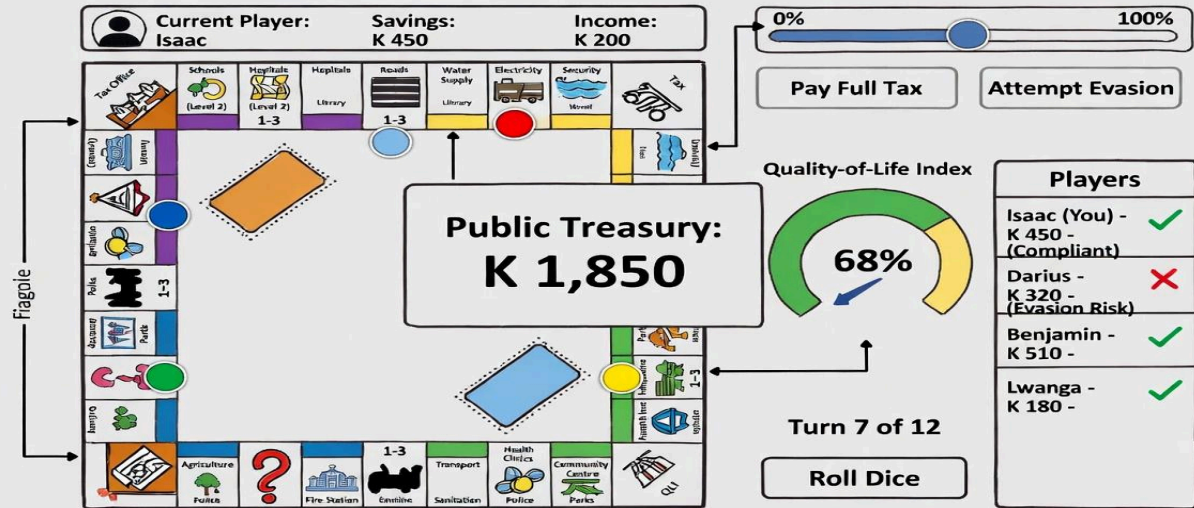


Figure 1.4.4: Tax-opoly Main Game Interface (Godot 2D Wireframe)

Tax-opoly Civic Risk / Public Good Card Popup

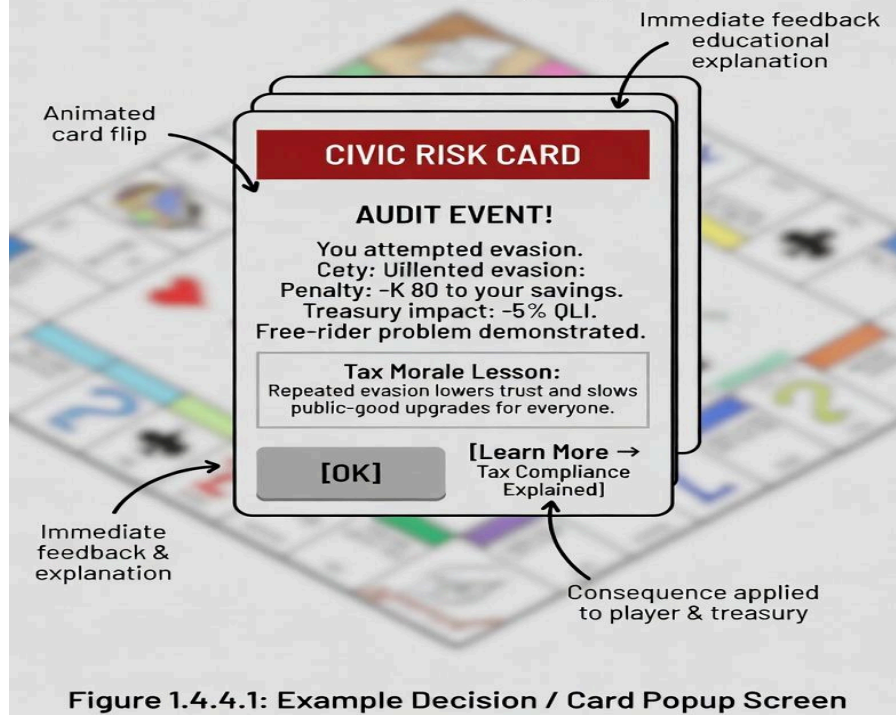


Figure 1.4.4.1: Example Decision / Card Popup Screen

Game logic design

Tax-opoly deliberately adopts the familiar structure and core loop of the classic Monopoly board game but re-themes every mechanic around taxation, public goods provision, and civic responsibility. This design choice directly addresses the critique in the Literature Review that Monopoly teaches purely individualistic, zero-sum economics. By contrast, Tax-opoly transforms the experience into a collaborative public-goods simulation, making the free-rider problem, tax morale, and collective investment tangible and experiential for players.

The game is strictly turn-based (4 players maximum) with authoritative state management on the host peer, ensuring fairness and educational integrity. All rules are enforced in the GameState and TurnManager singletons defined in the Class Diagram.

The table below outlines some design adaptations that allow Tax-opoly to draw inspiration from the classic monopoly but tailor it for tax compliance education.

Monopoly Element	Tax-opoly adaptation	Educational purpose
Board and Property Squares	16 fixed public-service spaces (Schools, Hospitals, Roads, Water Supply, Security, Library, Electricity, Transport, Agriculture, Health Clinics, Police, Fire Station, Parks, Community Centre, Sanitation, Tax Office)	Players “land” on services and vote to upgrade them using the shared treasury, demonstrating how taxes fund visible public goods.
Buying Properties	Collective investment vote: if the Public Treasury has sufficient funds, all players approve or reject an upgrade (animated on the board)	Illustrates the multiplier effect of collective tax contributions and the free-rider dilemma.
Passing “Go”	Every full lap around the board = automatic monthly income (salary)	Represents real-world income that is then subject to taxation decisions.

Chance and Community Chest Cards	Two separate decks: Civic Risk Cards (audits, penalties, free-rider events) and Public Good Cards (bonuses, infrastructure windfalls)	Provides immediate, contextual explanations of tax compliance consequences and civic lessons.
Tax / Rent Payment	On landing on a space or via card: player chooses full tax, partial tax, or evasion attempt via slider	Core decision point that teaches the slippery-slope framework and tax morale (Kirchler, 2007).
Jail	Temporary “Audit Lock” – player cannot move or earn income until penalty is paid	Visual penalty for repeated evasion, reinforcing deterrence and trust erosion.
Winning Condition	The game ends after 12 fixed rounds or when QLI reaches a target threshold. Final score balances personal savings and contribution to community QLI	Shifts focus from pure individual wealth (Monopoly) to balanced individual + collective benefit.

There are a sequence of events that a player is allowed to experience in each turn of the game. Each turn follows the same structured flow (implemented via the TurnManager singleton and RPC calls to the host):

1. Income Phase – Player receives monthly salary (displayed in HUD).
2. Movement Phase – Token advances automatically or via dice roll (simple 2D animation on the board).
3. Tax Decision Phase – Player lands on a public-service space or draws a card and uses the tax slider (0 % – 100 %) to declare full tax, partial tax, or attempt evasion.
4. Resolution Phase – Host authoritatively processes the action:
 - Tax paid → added to the Public Treasury.
 - Evasion attempt → Civic Risk Card draw + possible audit penalty.
 - Public Good Card → possible bonus to treasury or QLI.

5. Investment Phase – Optional group vote to upgrade the landed service if treasury balance allows (visible upgrade animation + QLI recalculation).
6. State Synchronization – Host broadcasts updated Player data, Treasury balance, upgrade levels, and QLI to all clients via MultiplayerSynchronizer and RPCs.
7. Feedback & Education – Immediate colour-coded message and card popup explain the outcome (e.g. “Tax Paid – +QLI for everyone!” or “Audit Failed – Free-rider problem demonstrated”).

The Quality-of-Life Index (QLI) is recalculated after every contribution or upgrade using a simple weighted formula stored in GameState:

$$\text{QLI} = [(\text{Total treasury contributions} / \text{Max possible treasury}) \times 0.6] + [((\text{Number of upgraded services}) / 16) \times 0.4]$$

This formula ensures players immediately see the link between individual tax decisions and collective societal benefit.

Data persistence

Although the core gameplay is intentionally session-based and stateless, all dynamic game session data is addressed through a lightweight, browser-compatible persistence mechanism built directly into the Godot Engine. This approach fully respects the chosen design constraints of the project: no external database and minimal resource usage on low-spec devices.

- Player Names: Player names are captured once during the lobby/join phase using simple LineEdit fields on the Main Menu screen. Each name is stored in the Player class instance and displayed throughout the session in the HUD and player sidebar. At game end, the names are automatically carried forward into the final score record.
- High Scores: After the game concludes (either after 12 rounds or when the QLI target is reached), the system calculates the final Quality-of-Life Index, total tax contributed by

each player, and an overall compliance score. These values, together with the player names, are saved locally on the host's browser.

- **Technical Implementation:** Persistence is achieved using Godot's built-in ConfigFile class, which stores data as a simple .cfg file in the browser's IndexedDB (HTML5 export). No additional plugins or external services are required keeping the game simple and easy to load ensuring that players are served with the data in the shortest possible time even with low bandwidth networks.

CONCLUSION

This document has presented a clear and practical system design for “Tax-opoly: The Public Good Game” using Godot Engine. The chosen client-server architecture i.e. host as authority, MVC pattern adapted to Godot nodes, and detailed UML diagrams ensure the game is modular, secure for turn-based play, and easy to implement. These decisions directly support the main goal which is to show Zambian students how paying taxes builds shared public goods e.g. schools, roads and hospitals.